

交互、通信选讲

清华大学 刘海峰

February 7, 2026

当然，也有些交互题是为了限制某些量、函数的调用次数从而出成交互的形式。

AGC044D

交互库有一个长度不超过 L 的字符串 S ，其字符集为所有大小写字母加数字。

你每次可以询问交互库一个长度不超过 L 的字符串 T ，交互库会返回 S, T 的编辑距离。

定义两个字符串的编辑距离为：可以对第一个字符串做插入字符、删除字符以及修改字符使得其变成第二个字符串所需的最小操作数。

$L = 128$ ，询问次数限制：850。

编辑距离是很难刻画的，考虑通过编辑距离能获得什么信息。

首先如果 S, T 的编辑距离为 $|S| - |T|$ ，那说明 T 一定是 S 的子序列。

编辑距离是很难刻画的，考虑通过编辑距离能获得什么信息。

首先如果 S, T 的编辑距离为 $|S| - |T|$ ，那说明 T 一定是 S 的子序列。

同时注意到我们可以询问 L 个相同的字符组成的字符串得出这个字符的出现次数。

然后分治归并排序即可。

IOI2018 Highway Tolls

给定一张 n 个点 m 条边的无向连通图，交互库隐藏了 S, T 两个点，同时给定两个正整数 $A, B (A < B)$ ，你每次可以将一些边的权值标为 A 、剩余的边权值标为 B ，然后交互库会返回 S, T 之间的最短路。

你需要通过不超过 50 次询问确定 S, T （不关注顺序）。

$2 \leq n \leq 9 \times 10^4, 1 \leq m \leq 1.3 \times 10^5$ 。

CF1764G3

交互库有一个长度为 $n(3 \leq n \leq 1024)$ 的 1 到 n 的排列 p , 你可以做至多 M 次询问, 每次询问给定 (l, r, k) , 交互库会返回 $[\lfloor \frac{p_l}{k} \rfloor, \lfloor \frac{p_{l+1}}{k} \rfloor, \dots, \lfloor \frac{p_r}{k} \rfloor]$ 中有多少个不同的数。

然后你需要确定一个 i 满足 $p_i = 1$ 。

满分限制: $M = 20$ 。

部分分: $M = 21, M = 30$ 。

注意到当询问 $k = 2$ 的时候, 大多数数 x 都存在 y 满足

$$1 \leq y \leq n, x \neq y, \lfloor x/y \rfloor = \lfloor y/2 \rfloor.$$

但 1 不存在, n 为偶数的时候 n 也不存在。

于是这给我们提供了解题思路。

考虑如果依次询问 $(1, x, 2)$, $(x + 1, n, 2)$, 那么我们可以得出左边没有被匹配的数的数量和右边没有被匹配的数的数量。

如果如果左边不等于右边则区分了 1 可能所在的位置。

如果等于说明 1 和 n 一个在左边, 一个在右边。

考虑判断 n 在哪边, 其实很简单, 只要全部除以 n , 然后看不同的数的数量即可。

于是可以直接二分, 这样算一下大概是 $2 \log n + 1 = 21$ 次。

考虑如何优化掉最后一次，不妨假设 n 一定不在 $[l, r]$ 内，既然能确定到这个区间，那么显然我们已经询问过 $(1, l-1, 2)$ 和 $(1, r, 2)$ 了。

那么根据结果，我们能够判断除了 1 之外的那个数是和 $[1, l-1]$ 中的数匹配还是和 $[r+1, n]$ 中的数匹配。

于是就可以只一次询问了，于是就省掉了一次。

IOI2025 纪念品

有 n 种纪念品（下标从 0 开始），第 i 种纪念品的价格为 P_i ，供应量为正无穷。保证 $P_0 > P_1 > \cdots > P_{n-1} > 0$ 。你现在只知道 n, P_0 。

每次你可以给定一个正整数 x ，然后交互库将执行以下流程：

- 对于 $i = 0 \sim n-1$ ，如果 $x \geq P_i$ ，则购买一次物品 i ，然后将 x 减去 P_i 。

要求每次操作至少购买一件商品。

之后，交互库会返回你购买了哪些物品，以及剩余了多少钱。

你需要调用至多 5000 次购买函数，使得第 i 种物品在所有调用中被购买了恰好 i 次（这意味着第 0 种物品不能够被购买）
 $2 \leq n \leq 100$ 。

直观上来看，第一次我们肯定要询问 $P_0 - 1$ 。

假如我们现在询问出某 m 个商品的价值和为 k ，那么接下来询问 $\lfloor \frac{k-1}{m} \rfloor$ ，可以发现，除非这次只够买到了第 n 件商品，否则接下来显然能购买至少一件商品，且购买到商品的最小编号肯定会变大。于是按照该做法我们可以获取第 n 个商品的价值。然后我们可以把所有询问出的结果减掉第 n 个商品的价值。

于是可以这么做，设 B_i 表示某次购买到的一组商品集合，满足编号最小的商品编号为 i ，如果目前没询问到相应的组合，那么 B_i 为空。

第一次询问 $P_0 - 1$ ，然后接下来每次找到最大的非空的 B_i 并按照刚才的方式进行询问，稍微处理一下即可，最后达到的效果是通过不超过 i 次购买得出了商品 i 的价格。

QOJ 10182

给定一棵 n 个点的有根树，树上每个点都有一个正整数权值（但你不知道），保证所有节点的权值都小于等于 n 且互不相同，保证父亲节点的权值小于儿子节点的标号。

你现在需要对交互库做若干次询问，每次给定两个点的编号，交互库会返回这两个点的权值哪个更大。

你需要通过至多 $1.4 \lfloor \log_2 P \rfloor$ 次操作确定答案，其中 P 为这棵树的所有可能的赋权方案数。

Finger search

我们知道交互题对交互次数的限制是比较严的，这点在普通的 OI 题中难以通过时间限制实现区分，所以需要格外小心。

假设交互库有一个长度为 n 的 01 序列，里面有恰好 m 个位置是 1 。

你每次可以询问交互库一个区间，交互库会返回这个区间中是否有 1 （或返回 1 的个数），我们希望用尽量少的次数确定所有 1 的位置。

如果直接二分下一个位置，复杂度是 $O(m \log n)$ 的，如果考虑直接扫一遍，复杂度是 $O(n)$ 的。都不太优美。

Finger search

我们知道交互题对交互次数的限制是比较严的，这点在普通的 OI 题中难以通过时间限制实现区分，所以需要格外小心。

假设交互库有一个长度为 n 的 01 序列，里面有恰好 m 个位置是 1。

你每次可以询问交互库一个区间，交互库会返回这个区间中是否有 1（或返回 1 的个数），我们希望用尽量少的次数确定所有 1 的位置。

如果直接二分下一个位置，复杂度是 $O(m \log n)$ 的，如果考虑直接扫一遍，复杂度是 $O(n)$ 的。都不太优美。

考虑从前往后一个一个找的过程中，如果当前位置和下一个位置的差为 d ，我们希望有一个 $O(\log d)$ 的算法。

根据 Jensen 不等式，在所有点之间的距离都是 $\frac{n}{m}$ 的时候复杂度是最劣的，此时是 $O(m \log \frac{n}{m})$ 。

比如说我们找到一个位置之后可以倍增寻找下一个位置，枚举一下倍增上界。

还有常数更小的处理方法：我们每次先询问一下接下来 n/m 个位置中是否有 1，如果没有直接跳过，否则就递归进去做。不过在不知道 m 的情况下最好还是写枚举倍增上界的做法。

当然还有另外一种处理方法，就是考虑类似线段树一样递归去计算。

不过基本上表现都不如该算法。

考虑估计 $\log \binom{n+m}{m}$ 的大小，其中 $n \geq m$ 。

注意到 $\log n! = n \log n + O(n)$ ，所以最终结果大致为 $O((n+m) \log \frac{n}{m})$ ，与 Finger search 的复杂度差不多。

所以对于本题，直接自底向上维护当前权值的相对大小即可。使用 Finger search 进行合并即可。

UNR6 神隐

交互库有一个 n 个节点的树，树的边是有顺序的。你每次可以询问一个长度为 $n - 1$ 的 01 串，然后如果第 i 个位置是 1 则交互库会保留树的第 i 条边，否则会删掉第 i 条边，然后交互库会返回当前情况下形成的连通块集合。你需要通过不超过 $limit$ 次询问确定这棵树。

满分： $n = 2^{17}$, $limit = 20$ 。

部分分： $n = 2000$, $limit = 14/22$ 以及 $n = 2^{17}$, $limit = 34$ 。

首先不难得出一种询问次数 $2 \log n$ 的做法，对于 $i = 1 \sim \log n$ ，在第 i 次询问中，我们询问边的编号二进制下第 i 位为 1 的边，在第 $i + \log n$ 次询问中，我们询问边的标号二进制下第 i 位为 0 的边。

那么如果两个点 x, y 在恰好 $\log n$ 次中都处于一个连通块内，那么 x, y 之间存在连边。

证明是显然的。

接下来分成了两个部分，一是如何卡交互次数，二是如何优化复杂度。

先看怎么卡交互次数，注意到我们上述做法正确的原因是如果 x, y 之间的最短路径大于 1，那么 x, y 上的边的编号交起来的 popcount 不等于并起来的 popcount。

于是可以考虑构造若干个大小为 $[1, limit] \cap Z$ 的子集，满足所有集合大小相等且互不相同，那么只要有超过一个那么交起来的大小一定会改变。

注意到最多能构造 $\binom{20}{10} = 184756$ 个，所以可以接受。

接下来考虑如何优化复杂度。

考虑剥叶子，问题变成了如何判断一个点是不是叶子。

做法是简单的，考虑如果一个点在 10 个询问中都处于单点的状态，那么这个点就是叶子，然后我们就可以不断删叶子，从而得到树的一个拓扑序。

然后考虑如何用这个拓扑序来还原这棵树。

注意对于一个连通块，拓扑序最大的点一定是其余的点的祖先，而对于每个点，一定存在一个连通块满足该连通块中拓扑序最大的点是该点的父亲节点，所以直接做即可。

CF2164G

交互库有一个 n 个点的无根树，你每次可以询问交互库一个排列 p ，交互库会返回一个长度为 n 的序列 a ，其中 a_i 表示有多少对 (x, y) 满足 $1 \leq x < y \leq i$ 且 p_x, p_y 有边相连。

$3 \leq n \leq 5 \times 10^4$ ，要求询问次数不超过 31 次。

思考一下我们能够通过这个问出些什么。

首先对得到的 a 序列做差分，我们可以问出 $p_1 \dots p_{i-1}$ 中有哪些点和 p_i 有边相连。

于是我们问一下 p ，再问一下 $reverse(p)$ ，我们就可以得到树上每个节点的度数。

当然只有这个还不能做这题。

我们先考虑我们可以通过什么信息能确定一棵树。
一种想法是确定每条边，不过放到这题里很不现实。
实际上我们也可以剥叶子。

具体而言，我们可以维护每个点的度数以及每个点的所有邻居的编号和，然后每次找一个度数为 1 的点剥掉即可。

考虑如何求和一个点有连边的点的编号之和，一种显然的方法就是倍增，数二进制下每一位出现了多少个。

具体来说，枚举 i ，然后把二进制下第 i 位为 1 的排成一个序列 A ，为 0 的排成一个序列 B 。

第一次询问交互库 AB ，第二次询问交互库 BA ，然后减一下就可以得到答案。

复杂度是 $2 \log n + 1$ ，因为问度数可以随便找一个然后 reverse 一下。

但交互次数多了一点。

这里有一个比较经典的 trick, 就是 $f(x) = \frac{x}{\ln x}$ 在 $(1, +\infty)$ 上最小值在 $x = e$ 处取到, 并且有 $f(2) > f(3)$ 。
也就是说这种情况下采取三进制其实是更优的。
用三进制复杂度是 $3 \log_3 n + 1$ 次, 可以通过。

IOI2020 mushroom

有 $n (n \leq 2 \times 10^4)$ 个蘑菇，每个蘑菇有 A,B 两种类型，保证第一个蘑菇是 A 类型。

现在有一台机器，每次可以将一些蘑菇排成一个序列放进去，机器会返回一个数 x ，表示有多少对相邻的蘑菇类型不同，例如放入 ABBA 会返回 2。

你现在想知道 A 类型的蘑菇总共有多少个。你需要保证机器调用次数不超过 226 次，且放入机器的蘑菇总数不超过 10^5 。

假如我们现在已经有一定量的 A,B 类型的蘑菇了。

假设我们当前有 x 个 A 类型的蘑菇, y 个 B 类型的蘑菇,不妨假设 $x \geq y$ 。

那么我们可以 $Ap_1Ap_2Ap_3, \dots, Ap_x$ 这样去询问, 可以得出 $p_1 \dots p_{x-1}$ 中有多少个 A 类型的蘑菇, 以及通过奇偶性来判断 p_x 的类型。

这样我们就可以做到 $O(\sqrt{n})$ 次调用了, 虽然距离目标还是有点远。

当 A, B 数量比较少的时候，一次询问获取一个蘑菇类型性价比不如获取两个蘑菇类型，于是可以选择问 $ApAq$ ，这样就可以知道 p, q 两个蘑菇的类型了。

设一个阈值 B ，询问次数小于等于 B 时采取第一种策略，大于 B 时采取第二种策略。

这样可以获得 92 分左右的高分。

询问次数小于等于 B 的部分有没有更好的做法。

首先每次询问通过奇偶性得到的那一个几乎没有优化的可能。考虑前面的部分能不能优化。

考虑第一次问 $AaAbA$ ，如果 a, b 两个蘑菇类型相等，那么就可以直接确定了。否则相当于是 a, b 两个蘑菇类型不同。

那么接下来问 $AaABbB$ 结果只有可能是 1 或 5。于是这里就有操作的空间了。

下一次直接询问 $AaABbBcB$ ，那么就可以获得 a, b, c 的具体类型。

于是这样相当于实现了两个问三个。

应用于本题再精细卡常可以获得满分。

实际上这个 2 问 3 有大量的优化空间。

其实这个问题本质上就是说，交互库有 n 个 01 变量，你每次给定 n 个 $\pm 1, 0$ 的权值，交互库会返回对应的 01 变量乘这一个值得到的答案。

实际上这个问题询问次数可以做到 $O(\frac{n}{\log n})$ 。

具体而言，假设我们现在存在一个 $a \times b$ 的矩阵 X ，其中 $X_{i,j} \in \{0, \pm 1\}$ ，满足按照这个矩阵询问可以获取前 b 个 01 变量的具体值。

那么考虑每次迭代令

$$X \rightarrow \begin{pmatrix} X & -X & I_a \\ X & X & 0 \end{pmatrix}, b \rightarrow 2b + a, a \rightarrow 2a$$

首先我们把第 i 行和第 $i + a$ 行相加通过奇偶性可以确定最后 a 个数，然后确定完之后就可以直接做了。

询问次数大致为 $O(\frac{n}{\log n})$ 。

实际上还可以进一步拓展，考虑每次我们询问给定的权值只可能是 01 的情况。

首先可以将 X 拆成两个矩阵相减，这样就转化成了第一个问题，但常数翻了一倍。

考虑令 Y 表示该解法的询问矩阵，设其规模为 $a \times c$ 。
那么每次迭代令

$$Y \rightarrow \begin{pmatrix} Y & X_1 \\ Y & X_2 \end{pmatrix}, c \rightarrow c + b$$

其中 X_1, X_2 是 $a \times b$ 规模的 01 矩阵，满足 $X_1 - X_2 = X$ 。
这样就可以获得一个常数更小的做法。

上述内容参考自 <https://www.luogu.com/article/1dzympve>

ICPC2025 沈阳站 C

给定 $n, m (n \leq m)$, 保证 $1 \leq n \times m \leq 10^6$ 。

第一次运行你将会收到 n 个互不相同的 $1 \sim m$ 中的数，你需要对每个数构造一个到长度为 $3m$ 且 $1 \sim m$ 恰好出现三次的序列。

随后, 交互库会对于每个序列, 构造一个 $\{1, 2, \dots, m\} \rightarrow \{0, 1\}$ 的映射并能将这个序列中的每个数作用于这个映射, 保证作用完之后序列不是全 0 或者全 1 序列。

第二次运行你将会收到 n, m 以及这 n 个 01 序列, 你需要确定这些 01 序列对应的原来的数是多少。

考虑剩余的部分怎么构造。

JOIST 2024 Spy 3

有一个 n 个点（编号从 0 开始） m 条边的无向图，每条边都有一个正整数权值。

Alice 有这张图的完整信息，而 Bob 手里的图中有 k 条边的权值缺失了（剩余的信息 Bob 也全知道），Alice 知道 Bob 手里缺失哪些边的信息。

Bob 现在想要知道 0 到 $T_1, T_2, \dots, T_Q$ 的最短路 (只需要找出一条最短路即可, 不要求出权值), 并且 Alice 也知道 $Q, T_1 \dots T_Q$ 。

Alice 需要给 Bob 传输尽量少的比特, 使得 Bob 能成功找出相应的最短路。

$$2 \leq n \leq 10^4, 1 \leq m \leq 2 \times 10^4, 1 \leq Q \leq 16, 1 \leq K \leq 300, \text{边权} \leq 10^{12}。$$

满分限制：1560 bits。

注意到最短路会构成一个树形结构，我们只需要把树的信息想办法发送过去即可。

IOI2025 恐龙大迁徙

有一个 n 个点的有根树，保证根节点为 1 且所有点的父亲编号都小于该节点的编号。

第一次运行时你将获取 n ，然后依次收到每个节点的父亲，在每收到一次之后需要发送一个 $0 \sim Z$ 之间的数。你需要保证发送的非 0 数的数量不超过 M 。

第二次运行的时候你将会获取 n 和你发送的数的序列，你需要确定这棵树的一对直径的端点（即找到一组 x, y 使得 x, y 的树上距离最大）。

满分限制: $Z = 4, M = 8$ 。

部分分: $Z = 5, M = 50, \quad Z = 4, M = 30, \quad Z = 4, M = 11。$

如果当前直径端点为 (x, y) ，那么结果只可能是 $(x, n - 1/n), (n - 1/n, y), (n - 1, n), (x, y)$ ，一共六种可能，所以 $Z = 5$ 其实是好做的。

然后对于前面的情况，考虑分块传递信息，用 dp 构建决策即可。

$\frac{33 \times 34}{2} = 561 = 140 \times 4 + 1$, 之后每次操作分别除以 $[9, 5, 3, 3]$ 。

可以通过计算得出最坏情况最后剩余的 A, B 集合大小都为 5。

上述的加密方法是对称加密，即可以用一个密钥进行加密解密，缺点刚才也提到了。

于是就有了非对称加密，非对称加密同时拥有公钥和私钥，公钥用于加密，私钥用于解密，即便知道了公钥也无法在有效时间内得出私钥。

一种想法是 Alice 直接生成一对公钥和私钥，把公钥告诉 Bob，这样 Bob 就可以用公钥加密信息并发送给 Alice。

P, k_1, x^{k_1} 也无法还原出 x 。

那么我们有 $x_k = (g^b h^k \times h^{-k})^{a_k} = g^{a_k \times b}$ ，于是 Bob 就知道了第 k 个信封的密钥，直接解密即可。

之后 Bob 再通过 OT 得到 B_Y 的值, 然后 Alice 对于所有的 X, Y , 求出 $C_{f(X,Y)}$ 按照 A_X, B_Y 进行对称加密得到的结果, 然后打乱并发送给 Bob。

混淆电路

考虑现在有一个数字电路，有些门是 Alice 进行输入，有些门是 Bob 进行输入，有些门是对某两个门进行逻辑运算，Alice 和 Bob 都知道整个电路的形态，Alice 需要知道电路中某个指定门的输出结果（Bob 也知道这个指定门）。

我们都知道，数字电路的功能是强大的，可以实现加法器、乘法器以及各种复杂的运算，因此很多协同计算相应函数且不泄露信息的问题都可以迎刃而解。

谢谢大家